

**Department of Artificial Intelligence and Machine Learning**

Date: 16/12/2025	Test - II	Max. Marks:50
Semester: V	UG	Duration: 1 $\frac{1}{2}$ Hrs
Course Title: Machine Learning Operations [MLOps]		Course Code:AI254TA

Sl no	QUESTIONS – PART B	M	BT	CO
1a.	Discuss the need of reproducibility and Auditability in building MLOps pipeline.	4	2	5
1b.	Explain how automated testing pipelines CI/CD reduce deployment failures in ML projects. Give example	6	3	4
2a.	What are the two different types of model deployment approaches used in MLOps? Explain with suitable examples	4	2	2
2b.	An autonomous vehicle company uses ML models for object detection. Hackers attempt to manipulate street signs to mislead the vehicle's perception system. Explain how machine learning security and risk mitigation measures can safeguard production systems in such safety-critical applications.	6	4	3
3a.	Elaborate the key considerations when preparing machine learning models for production deployment.	6	4	3
3b.	"For batch scoring, the scaling deployments can be more complex". Analyse the statement and discuss the two types of strategies to distribute the computation.	4	2	3
4a.	A healthcare analytics company uses an ML model to predict early signs of cardiac risk based on live patient vitals. To upgrade to a new model version without interrupting critical monitoring, the team wants to use a Blue-Green deployment approach. Explain how Blue-Green deployment ensures safe and zero-downtime release of the updated model. Enumerate the potential risks if real-time monitoring traffic suddenly switches to the new model and provide the steps mitigate the risks.	5	4	3
4b.	Describe the significance of containerization technologies in MLOps and evaluate the challenges faced while working with Docker environments.	5	4	5
5	A smart-city project deploys an ML-based traffic prediction system that uses live sensor data, historical traffic flow, and weather conditions to optimize signal timings and reduce congestion. The model is developed in a controlled environment and then pushed to production using MLOps practices such as automated data pipelines, model validation, deployment workflows, and real-time monitoring. Examine the risks involved when migrating this ML system from development to production, focusing on data reliability issues, model drift, integration with IoT devices, latency challenges, and security threats. Justify the importance of selecting a suitable runtime environment to ensure low-latency, scalable, and reproducible ML operations, Justify your solution.	10	4	3



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

M-Marks, BT-Bloom's Taxonomy Levels, CO-Course Outcomes

Marks Distribution	Particulars	CO1	CO2	CO3	CO4	CO5	L1	L2	L3	L4	L5	L6
	Max Marks CIE & Quiz	-	4	31	6	9	-	12	6	32		



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Date:	Test - II	Max. Marks:50
Semester:V	UG	Duration: 1 $\frac{1}{2}$ Hrs
Course: Machine Learning Operations [MLOps]		Code: AI254TA

SCHEME & SOLUTIONS



S No	Solutions with Scheme	Marks
1a	Need for Reproducibility in MLOps Reproducibility means the ability to recreate the same model, with identical performance, using the same data, code, environment, and configuration. a) Ensures Consistent Model Behaviour b) Simplifies Debugging and Error Analysis c) Essential for Collaboration d) Facilitates Compliance and Validation Need for Auditability in MLOps a) Regulatory and Legal Compliance b) Traceability of Model Decisions c) Ensures Ethical and Responsible AI d) Enables Better Risk Management	4
1b	Automated testing pipelines such as Jenkins-based CI/CD significantly reduce deployment failures in ML projects by ensuring that every change to code, data, or configuration is verified before reaching production. Jenkins automates the entire workflow—from data validation and feature checks to model testing—so issues are detected early instead of after deployment. By running unit tests, integration tests, data schema checks, and performance evaluations automatically, the pipeline ensures that models meet accuracy thresholds and do not break due to data drift, coding errors, or dependency mismatches. Automation also encourages frequent and smaller deployments, making debugging faster and less complex because only small changes are introduced at a time. Additionally, Jenkins orchestrates environment replication using Docker or virtual environments, reducing inconsistencies between development and production. Overall, automated CI/CD pipelines minimize human error, speed up feedback cycles, ensure reproducibility, and prevent the common failures caused by outdated data, incorrect preprocessing, or incompatible model versions—leading to more reliable and stable ML deployment.	6
2a	Batch scoring processes entire datasets periodically, while real-time scoring handles one or few requests instantly. Batch scoring, where whole datasets are processed using a model, such as in daily scheduled jobs. Real-time scoring, where one or a small number of records are scored, such as when an ad is displayed on a website and a user session is scored by models to decide what to display.	4
2b	i. Vulnerabilities: Adversarial examples on street signs, sensor spoofing. ii. Security Measures: Robust training with adversarial examples, secure sensor inputs, real-time anomaly detection.	6



	iii. Risk Mitigation: Redundant sensors, model ensemble, regular updates, rollback mechanisms. iv. Example: Detecting manipulated signs and switching to backup models to avoid accidents.	
3a	i. Runtime Environment ii. Model Risk Evaluation iii. Quality Assurance iv. Reproducibility and Auditability v. Security vi. Risk Mitigation	5
3b	For batch scoring, the scaling deployments can be more complex” means that when an ML model processes large volumes of data in batches (e.g., millions of records at once), it requires significant computational resources. Unlike real-time scoring (which deals with one request at a time), batch jobs: • consume heavy CPU/GPU resources, • must process huge datasets within strict deadlines, • often run on distributed systems, • need proper coordination to avoid bottlenecks. Therefore, scaling such workloads becomes complex because the system must efficiently divide large datasets, parallelize processing, and manage failures or slow nodes. Horizontal Scaling (Distributed Batch Processing) Horizontal scaling means distributing the batch job across multiple machines or nodes. • The dataset is split into partitions/chunks. • Each node processes a subset of data in parallel. • Frameworks like Apache Spark, Hadoop MapReduce, Dask, Ray are commonly used. • Results from each node are merged at the end. Vertical Scaling (Scaling Up on a Single Machine) Vertical scaling means increasing the capacity of a single system instead of distributing the work. • Add more CPU, RAM, or GPU resources to one powerful machine. • Run the entire batch scoring process locally. • Suitable for mid-sized datasets that fit into memory	5



4a	Blue-Green deployment is a release strategy where two identical environments—Blue (current production) and Green (new version)—run in parallel. The Blue environment hosts the currently running ML model. The Green environment contains the updated model version. Traffic is directed to only one environment at a time. After validating the new version in Green, traffic is gradually switched from Blue to Green. - Both model versions run simultaneously, so updating the model does not stop real-time monitoring. - Incoming patient-vital streams are served by Blue while Green is prepared, reducing downtime to zero. - If any issue occurs in the Green model after switching, traffic can be instantly rolled back to Blue. - Enables safe validation of the new cardiac risk model with real data before exposing it to live monitoring. The team should follow these steps before switching real traffic: Shadow Testing – Run the Green model on real live vitals in parallel with Blue, but do not expose predictions to clinicians. Compare outputs for accuracy and latency. A/B Metrics Comparison – Ensure prediction stability, false-alarm rate, precision/recall, and alert patterns match clinical expectations. Load & Latency Testing – Validate that Green handles peak streaming load with acceptable response times. Schema and API Compatibility Check – Ensure the new model accepts and returns data in the correct format. Rollback Plan Ready – Ensure switching back to Blue is fast and automated if an issue is detected.	4
4b	Role of Containerization in MLOps / ML Model Deployment Containerization packages an ML application along with its dependencies, libraries, runtime, and configuration into a single portable unit, ensuring consistent execution across environments. - Ensures Environment Consistency - Supports Scalable Deployment - Faster and Lightweight Deployment - Easy Versioning and Reproducibility - Better Portability - Simplifies CI/CD in MLOps Challenges Faced by Docker (Limitations & Issues) - Security Vulnerabilities - Resource Management Problems	6



	c) Networking Complexity d) Storage and Data Handling Challenges e) Large Image Sizes f) Difficulty in GPU Acceleration	
5	1. Risks in Migrating the ML System from Development to Production a) Data Reliability Issues b) Model Drift c) Integration Challenges with IoT Devices d) Latency Challenges e) Security Threats 2. Importance of Selecting a Suitable Runtime Environment a) Low-Latency Inference b) Scalability c) Reproducibility d) Stability and Maintenance 3. Real-World Example Example: Google Maps Traffic Prediction • Google deploys traffic prediction models on a combination of cloud servers and edge environments. • Edge deployment helps process sensor and GPS data closer to the user, ensuring ultra-low latency. • Scalable Kubernetes clusters handle workload spikes and ensure smooth, reproducible deployments.	10